

Machine Learning A (2026–2027)

Home Assignment 3

Ludovico Maria Spitaleri, DMH249

Contents

1	Optimal Classification (16 points)	2
1.1	Question 1	2
1.2	Question 2	2
1.3	Question 3	3
2	Logistic Regression (50 points)	3
2.1	Cross-entropy error measure (16 points)	3
2.1.1	Rewriting negative logarithmic likelihood as cross-entropy	3
2.1.2	Cross-entropy and logistic regression	3
2.2	Logistic regression loss gradient (24 points)	3
2.2.1	Gradient for -1 and 1	3
2.2.2	Gradient for 0 and 1	3
2.2.3	Influence of misclassified examples	3
2.3	Log-odds (10 points)	3
3	Sleep Well (34 points)	3
3.1	Data understanding and preprocessing (4 points)	3
3.2	Classification (30 points)	4
3.2.1	Logistic regression	4
3.2.2	Random forest	4
3.2.3	Nearest neighbor	5
4	Overfitting (optional, 0 points)	6

1 Optimal Classification (16 points)

1.1 Question 1

Let us map the original label space $Y \in \{0, 1\}$ to a new space $Y' \in \{-1, 1\}$ following the original probabilities table for X and Y .

We define our decision function based on the original probabilities:

$$\delta'(x) = P(Y' = 1 \mid X = x) - P(Y' = -1 \mid X = x)$$

The classification rule in the mapped space is:

$$h'(x) = \text{sgn}(\delta'(x))$$

Evaluating this for the given inputs:

$$\begin{cases} \delta'(0) = 0 - 1 = -1 \Rightarrow h'(0) = \text{sgn}(-1) = -1 & \text{if } x = 0 \\ \delta'(1) = 0.2 - 0.8 = -0.6 \Rightarrow h'(1) = \text{sgn}(-0.6) = -1 & \text{if } x = 1 \end{cases}$$
$$\Rightarrow h'(x) = -1$$

Mapping back to the original space Y , the optimal classification rule in the original space is $h(x) = 0$.

The expected risk (0-1 loss) is the probability of misclassification:

$$\begin{aligned} R &= P(h(X) \neq Y) \\ &= \sum_{x \in X} P(X = x) P(h(x) \neq Y \mid X = x) \\ &= \sum_{x \in X} P(X = x) P(Y = 0 \mid X = x) \\ &= 0.5 \cdot 0 + 0.5 \cdot 0.2 = 0.1 \end{aligned}$$

1.2 Question 2

In the case of $X = 0$, the probabilistic classifier still always predicts the correct label $Y = 0$, since $P(Y = 0 \mid X = 0) = 1$.

On the other hand, when considering $X = 1$ the expected risk is given by the probability of a mismatch between the classifier and the true label:

$$\begin{aligned} &P(h(x) \neq Y \mid X = 1) \\ &= P(h(x) = 0 \mid X = 1) P(Y = 1 \mid X = 1) + P(h(x) = 1 \mid X = 1) P(Y = 0 \mid X = 1) \\ &= 0.8 \cdot 0.2 + 0.2 \cdot 0.8 = 0.32 \end{aligned}$$

The total expected risk is therefore:

$$\begin{aligned} R &= P(h(X) \neq Y) \\ &= \sum_{x \in X} P(X = x) P(h(x) \neq Y \mid X = x) \\ &= 0.5 \cdot 0 + 0.5 \cdot 0.32 = 0.16 \end{aligned}$$

1.3 Question 3

No, it is not possible to find a better classifier than the best one previously found, that is the deterministic classifier defined in section 1.1. This is because by mapping the label space to $Y' = \{-1, 1\}$ the risk of this classifier corresponds to the Bayes risk, known to be the minimum over all possible measurable functions h .

$$h^{\text{Bayes}}(x) = \text{sgn}[\mathbb{E}_p(Y \mid X = x)]$$

2 Logistic Regression (50 points)

2.1 Cross-entropy error measure (16 points)

2.1.1 Rewriting negative logarithmic likelihood as cross-entropy

2.1.2 Cross-entropy and logistic regression

2.2 Logistic regression loss gradient (24 points)

2.2.1 Gradient for -1 and 1

2.2.2 Gradient for 0 and 1

2.2.3 Influence of misclassified examples

2.3 Log-odds (10 points)

3 Sleep Well (34 points)

3.1 Data understanding and preprocessing (4 points)

The library `pandas` is used to load the datasets from the csv files. The classes frequencies are then calculated directly using `numpy`.

```
counts_train = np.bincount(y_train)
freq_train = counts_train / counts_train.sum()
```

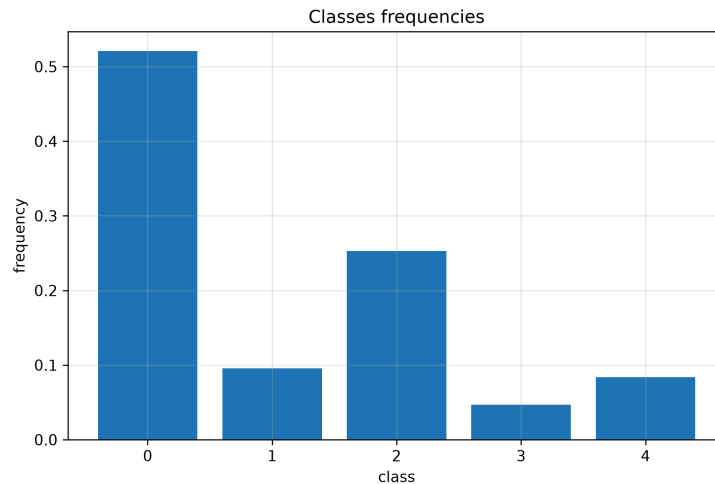


Figure 1: Classes frequencies bar chart

3.2 Classification (30 points)

3.2.1 Logistic regression

The `LogisticRegression` class from `sklearn` is used to fit a model to the training dataset using logistic regression. For both the train and test datasets, the error is computed using the appropriate labels.

Regularization was not performed.

```
lr = LogisticRegression(max_iter=150).fit(X_train, y_train)
lr_train_err = 1 - lr.score(X_train, y_train)
lr_test_err = 1 - lr.score(X_test, y_test)
```

Errors:

- Training set: 0.1496
- Test set: 0.0993

3.2.2 Random forest

The `RandomForestClassifier` class from `sklearn.ensemble` is used to fit a model to the training dataset using random forests. For both the train and test datasets, the error is computed using the appropriate labels. The out-of-bag error is also included.

Regularization was not performed.

```
rf = RandomForestClassifier(n_estimators=ntrees, oob_score=True).fit(X_train, y_train)
rf_train_err = 1 - rf.score(X_train, y_train)
rf_test_err = 1 - rf.score(X_test, y_test)
rf_oob_err = 1 - rf.oob_score_
```

Errors:

Trees	Training err	Test err	OOB err
50	0.0004	0.1132	0.1519

Trees	Training err	Test err	OOB err
100	$2.9652 \cdot 10^{-5}$	0.1101	0.1495
200	0	0.1099	0.1481

3.2.3 Nearest neighbor

The `KNeighborsClassifier` class from `sklearn.neighbors` is used to fit a model to the training dataset using k-nn classification.

First cross-validation is performed to determine the optimal value of k . Only odd values are used as candidates to avoid ties as much as possible. The range of values chosen is $[1, 200]$.

For each value of K , 5-fold cross validation is performed, then the candidate with the best average error across the folds is chosen as the optimal value of K .

```
ks = np.arange(1, 202, 2)
cv = KFold(n_splits=5)
k_errs = []
for k in ks:
    knn = KNeighborsClassifier(n_neighbors=k)
    cv_errs = 1 - cross_val_score(knn, X_train, y_train, cv=cv)
    k_errs.append(np.mean(cv_errs))
K_idx = np.argmin(k_errs)
K = ks[K_idx]
K_err = k_errs[K_idx]
```

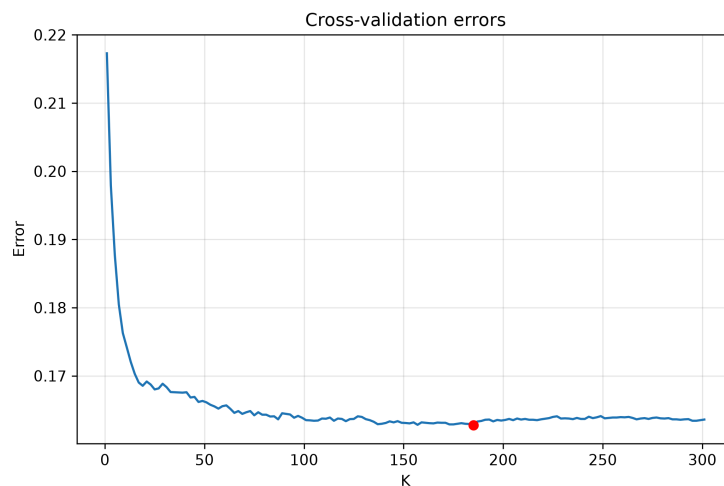


Figure 2: Cross validation errors, with the best value for K highlighted

For both the train and test datasets, the error is computed using the appropriate labels.

Regularization was not performed.

```
knn = KNeighborsClassifier(n_neighbors=K).fit(X_train, y_train)
knn_train_err = 1 - knn.score(X_train, y_train)
knn_test_err = 1 - knn.score(X_test, y_test)
```

Errors:

- Training set: 0.1503

- Test set: 0.0957

4 Overfitting (optional, 0 points)